Python Roadmap

Detailed Definitions with Multiple Examples for Every Topic

Complete Python Roadmap (Beginner → Advanced)

A structured, end-to-end path to learn Python — with free YouTube channels, websites, and **every topic explained with a detailed definition plus multiple examples.**

Best FREE Resources

YouTube Channels

Channel	Best For
freeCodeCamp.org	Full Python course (4-6 hr single video), beginner-friendly
Corey Schafer	OOP, decorators, file handling, virtual environments
Programming with Mosh	Clean, structured beginner-to-intermediate course
Telusko	Good for Indian learners, explains concepts slowly
CS Dojo	Simple explanations, great for absolute beginners
Tech With Tim	Projects + intermediate concepts (OOP, GUI, web scraping)
NeuralNine	Intermediate/advanced Python, automation, security
Sentdex	Data Science, Machine Learning with Python
Codebasics (Dhaval Patel)	Python for Data Analysis, Pandas, NumPy

Websites

Website	Use
python.org/doc	Official documentation (always accurate)
W3Schools.com/python	Quick topic reference + “Try it” editor
GeeksforGeeks.org	Topic-wise theory + practice problems
RealPython.com	In-depth articles, best for intermediate/advanced
HackerRank / LeetCode / CodeChef	Practice problems & coding tests
Programiz.com	Beginner tutorials with online compiler
Kaggle Learn	Free micro-courses for Python + Data Science
Exercism.org	Practice with mentor feedback

Roadmap Overview (Suggested Order)

1. Basics & Syntax
2. Variables & Data Types
3. Operators
4. Control Flow
5. Strings
6. Data Structures (List, Tuple, Set, Dict)
7. Functions
8. Object-Oriented Programming

9. Exception Handling
10. File Handling
11. Modules & Packages
12. Iterators, Generators, Comprehensions
13. Decorators & Closures
14. Regular Expressions
15. Dates & Time
16. Multithreading / Multiprocessing
17. Databases
18. Virtual Environments & pip
19. Popular Libraries
20. Projects & Practice

---

1. Basics & Syntax

Definition: Python is a high-level, interpreted, general-purpose programming language known for its clean, readable syntax. Unlike languages like Java or C++, Python uses **indentation** (whitespace) instead of curly braces `{}` to define blocks of code. It is dynamically typed, has automatic memory management, and supports multiple paradigms (procedural, object-oriented, functional).

Example 1 — Your first program

```
print("Hello, World!")
```

Example 2 — Indentation matters

```
if True:
    print("This is inside the if block")
    print("Still inside")
print("This is outside the block")
```

Example 3 — Comments

```
# This is a single-line comment
x = 5  # comment after code

"""
This is a multi-line comment,
also used as a docstring.
"""
```

Example 4 — Case sensitivity

```
name = "Alex"
Name = "Bob"
print(name, Name)   # Alex Bob (different variables)
```

---

2. Variables & Data Types

Definition: A **variable** is a named location in memory used to store a value. Python is **dynamically typed**, meaning you don't need to declare a type — Python figures it out automatically at runtime. Core built-in data types include `int`, `float`, `str`, `bool`, `complex`, `NoneType`, and container types like `list`, `tuple`, `set`, `dict`.

Example 1 — Basic variable assignment

```
name = "Alex"       # str
age = 25              # int
height = 5.9          # float
is_student = True     # bool
nothing = None         # NoneType
```

Example 2 — Multiple assignment

```
a, b, c = 1, 2, 3
x = y = z = 0
print(a, b, c, x, y, z)
```

Example 3 — Type checking and conversion

```
age = 25
print(type(age))          # <class 'int'>

age_str = str(age)        # int -> str
num = int("10")             # str -> int
price = float("9.99")     # str -> float
print(type(age_str), type(num), type(price))
```

Example 4 — Dynamic typing in action

```
value = 10
print(type(value))    # int
value = "now a string"
print(type(value))    # str
```

---

3. Operators

Definition: Operators are special symbols that perform operations on variables and values. Python supports **arithmetic**, **comparison (relational)**, **logical**, **assignment**, **bitwise**, **membership**, and **identity** operators.

Example 1 — Arithmetic operators

```
a, b = 10, 3
print(a + b)    # 13  addition
print(a - b)    # 7   subtraction
print(a * b)    # 30  multiplication
print(a / b)    # 3.333... division
print(a // b)   # 3   floor division
print(a % b)    # 1   modulus (remainder)
print(a ** b)   # 1000 exponentiation
```

Example 2 — Comparison operators

```
x, y = 5, 8
print(x == y, x != y, x > y, x < y, x >= y, x <= y)
```

Example 3 — Logical operators

```
a = True
b = False
print(a and b)   # False
print(a or b)    # True
print(not a)     # False
```

Example 4 — Membership and identity operators

```
fruits = ["apple", "banana"]
print("apple" in fruits)       # True
print("mango" not in fruits)   # True

x = [1, 2]
y = [1, 2]
print(x == y)   # True  (same value)
print(x is y)   # False (different objects in memory)
```

---

4. Control Flow

Definition: Control flow statements determine the order in which instructions are executed. This includes **conditional statements** (`if` , `elif` , `else`) and **loops** (`for` , `while`), along with `break` , `continue` , and `pass` .

Example 1 — if / elif / else

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

Example 2 — for loop with range()

```
for i in range(5):
    print(i)          # 0 1 2 3 4

for i in range(2, 10, 2):
    print(i)          # 2 4 6 8
```

Example 3 — while loop

```
count = 0
while count < 3:
    print("Count is", count)
    count += 1
```

Example 4 — break, continue, pass

```
for i in range(10):
    if i == 5:
        break          # exits the loop completely
    if i % 2 == 0:
        continue       # skips to next iteration
    print(i)

for i in range(3):
    pass   # placeholder, does nothing
```

Example 5 — Nested loops

```
for i in range(3):
    for j in range(2):
        print(f"i={i}, j={j}")
```

---

5. Strings

Definition: A **string** is an immutable sequence of Unicode characters, enclosed in single, double, or triple quotes. Python provides many built-in methods for searching, formatting, and transforming strings.

Example 1 — Creating and indexing strings

```
s = "Python Programming"
print(s[0])       # P
print(s[-1])      # g (last character)
print(s[0:6])     # Python  (slicing)
print(s[::-1])    # gnimmargorP nohtyP (reversed)
```

Example 2 — Common string methods

```
s = "  Python Programming  "
print(s.strip())          # removes leading/trailing spaces
print(s.upper())          # PYTHON PROGRAMMING
print(s.lower())          # python programming
print(s.replace("Python", "Java"))
print(s.strip().split(" "))   # ['Python', 'Programming']
```

Example 3 — String formatting

```
name, age = "Alex", 25

# f-string (recommended, Python 3.6+)
print(f"My name is {name} and I am {age} years old")

# .format() method
print("My name is {} and I am {} years old".format(name, age))

# % formatting (older style)
print("My name is %s and I am %d years old" % (name, age))
```

Example 4 — Useful checks

```
s = "Hello123"
print(s.isalpha())    # False (contains digits)
print(s.isdigit())    # False (contains letters)
print(s.isalnum())    # True
print(len(s))         # 8
print("Hello" in s)   # True
```

---

6. Data Structures

Definition: Python provides four built-in container types to store collections of data: - **List** — ordered, mutable, allows duplicates - **Tuple** — ordered, immutable, allows duplicates - **Set** — unordered, mutable, no duplicates - **Dictionary** — unordered (insertion-ordered since 3.7), mutable, stores key-value pairs

Example 1 — List operations

```
fruits = ["apple", "banana", "mango"]
fruits.append("grape")          # add to end
fruits.insert(1, "kiwi")          # insert at index
fruits.remove("banana")         # remove by value
print(fruits[0])                 # apple
print(fruits[-1])                # last item
print(len(fruits))               # 4
fruits.sort()                     # sorts in place
print(fruits)
```

Example 2 — Tuple operations

```
point = (10, 20)
print(point[0], point[1])   # 10 20
# point[0] = 99   # ERROR: tuples are immutable

x, y = point   # unpacking
print(x, y)
```

Example 3 — Set operations

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a.union(b))         # {1, 2, 3, 4}
print(a.intersection(b))  # {2, 3}
print(a.difference(b))    # {1}
a.add(5)
print(a)                   # {1, 2, 3, 5}
```

Example 4 — Dictionary operations

```
person = {"name": "Alex", "age": 25}
print(person["name"])       # Alex
person["city"] = "Chennai"    # add new key
person["age"] = 26             # update value

for key, value in person.items():
    print(key, ":", value)

print(person.get("country", "Not Found"))  # safe access with default
```

---

7. Functions

Definition: A **function** is a named, reusable block of code that performs a specific task. Functions help avoid repetition and organize code logically. Python supports default arguments, keyword arguments, variable-length arguments (`*args` , `**kwargs`), and anonymous (`lambda`) functions.

Example 1 — Basic function

```
def greet(name):
    return f"Hello, {name}!"

print(greet("Alex"))
```

Example 2 — Default and keyword arguments

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Alex"))                # Hello, Alex!
print(greet("Alex", "Hi"))           # Hi, Alex!
print(greet(name="Bob", greeting="Hey"))  # Hey, Bob!
```

Example 3 — *args and **kwargs

```
def add_all(*numbers):          # accepts any number of positional args
    return sum(numbers)

print(add_all(1, 2, 3, 4))     # 10

def print_info(**details):      # accepts any number of keyword args
    for key, value in details.items():
        print(f"{key}: {value}")

print_info(name="Alex", age=25)
```

Example 4 — Lambda (anonymous) functions

```
square = lambda x: x * x
print(square(5))    # 25

add = lambda x, y: x + y
print(add(3, 4))    # 7

# commonly used with map/filter
nums = [1, 2, 3, 4]
squares = list(map(lambda x: x**2, nums))
print(squares)      # [1, 4, 9, 16]
```

---

8. Object-Oriented Programming (OOP)

Definition: OOP is a programming paradigm built around **objects**, which combine data (**attributes**) and behavior (**methods**). Python's OOP model is built on four pillars: **Encapsulation**, **Inheritance**, **Polymorphism**, and **Abstraction**.

Example 1 — Class and object basics

```
class Car:
    def __init__(self, brand, speed):   # constructor
        self.brand = brand
        self.speed = speed

    def drive(self):
        print(f"{self.brand} is driving at {self.speed} km/h")

my_car = Car("Toyota", 120)
my_car.drive()   # Toyota is driving at 120 km/h
```

Example 2 — Inheritance

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} makes a sound")

class Dog(Animal):        # Dog inherits from Animal
    def speak(self):        # method overriding
        print(f"{self.name} barks")

a = Animal("Generic Animal")
d = Dog("Rex")
a.speak()   # Generic Animal makes a sound
d.speak()   # Rex barks
```

Example 3 — Encapsulation (private attributes)

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # double underscore = private

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
account.deposit(500)
print(account.get_balance())   # 1500
# print(account.__balance)     # ERROR: not directly accessible
```

Example 4 — Polymorphism

```
class Cat:
    def sound(self):
        return "Meow"

class Duck:
    def sound(self):
        return "Quack"

for animal in [Cat(), Duck()]:
    print(animal.sound())   # Meow, then Quack (same method call, different behavior)
```

Example 5 — Abstraction with abstract classes

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius ** 2

c = Circle(5)
print(c.area())   # 78.5
```

---

9. Exception Handling

Definition: Exception handling lets your program deal with runtime errors gracefully using `try`, `except`, `else`, and `finally` blocks, instead of crashing.

Example 1 — Basic try/except

```python
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

Example 2 — Multiple except blocks

```python
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ZeroDivisionError:
    print("Cannot divide by zero!")
except ValueError:
    print("That's not a valid number!")
```

Example 3 — else and finally

```python
try:
    x = 5 / 1
except ZeroDivisionError:
    print("Error occurred")
else:
    print("No error, result is", x)   # runs only if no exception
finally:
    print("This always runs")          # cleanup code
```

Example 4 — Raising custom exceptions

```python
def check_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

try:
    check_age(-3)
except ValueError as e:
    print("Error:", e)
```

---

10. File Handling

Definition: File handling refers to reading from and writing to files on disk. Python's `open()` function, ideally combined

with the `with` statement (context manager), handles this safely and automatically closes the file.

Example 1 — Writing to a file

```
with open("data.txt", "w") as f:
    f.write("Hello, Python!\n")
    f.write("This is line two.")
```

Example 2 — Reading a whole file

```
with open("data.txt", "r") as f:
    content = f.read()
    print(content)
```

Example 3 — Reading line by line

```
with open("data.txt", "r") as f:
    for line in f:
        print(line.strip())
```

Example 4 — Appending to a file

```
with open("data.txt", "a") as f:
    f.write("\nAppended line")
```

---

11. Modules & Packages

Definition: A **module** is a single `.py` file containing reusable functions/classes. A **package** is a directory of modules with an `__init__.py` file. Python's Standard Library ships with many built-in modules, and external packages can be installed via `pip`.

Example 1 — Using a built-in module

```
import math
print(math.sqrt(16))     # 4.0
print(math.pi)             # 3.14159...
```

Example 2 — Importing specific functions

```
from datetime import date
today = date.today()
print(today)
```

Example 3 — Aliasing imports

```
import numpy as np
arr = np.array([1, 2, 3])
print(arr)
```

Example 4 — Creating your own module

```
# In a file called mymath.py:
# def add(a, b):
#     return a + b

# In another file:
# import mymath
# print(mymath.add(3, 4))
```

---

12. Iterators, Generators & Comprehensions

Definition: An **iterator** is an object that implements `__iter__()` and `__next__()`, producing one value at a time. A

generator is a special function that uses `yield` to lazily produce values (memory-efficient). **Comprehensions** offer a compact syntax to build lists, sets, or dicts.

Example 1 — Iterators

```
nums = iter([1, 2, 3])
print(next(nums))   # 1
print(next(nums))   # 2
print(next(nums))   # 3
```

Example 2 — Generators

```
def count_up(n):
    for i in range(n):
        yield i

for num in count_up(3):
    print(num)   # 0 1 2
```

Example 3 — List comprehension

```
squares = [x**2 for x in range(5)]
print(squares)    # [0, 1, 4, 9, 16]

evens = [x for x in range(10) if x % 2 == 0]
print(evens)      # [0, 2, 4, 6, 8]
```

Example 4 — Dictionary and set comprehensions

```
squares_dict = {x: x**2 for x in range(5)}
print(squares_dict)   # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

unique_lengths = {len(word) for word in ["hi", "python", "go"]}
print(unique_lengths)  # {2, 6}
```

---

13. Decorators & Closures

Definition: A **closure** is a function that remembers variables from its enclosing scope even after that scope has finished executing. A **decorator** is a function that wraps another function to add extra behavior without modifying its source code, using the `@decorator_name` syntax.

Example 1 — Closures

```
def outer(msg):
    def inner():
        print("Message:", msg)
    return inner

my_func = outer("Hello!")
my_func()   # Message: Hello!
```

Example 2 — Basic decorator

```
def my_decorator(func):
    def wrapper():
        print("Before function runs")
        func()
        print("After function runs")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

Example 3 — Decorator with arguments

```
def repeat(func):
    def wrapper(*args, **kwargs):
        for _ in range(2):
            func(*args, **kwargs)
    return wrapper

@repeat
def greet(name):
    print(f"Hello, {name}!")

greet("Alex")   # prints twice
```

Example 4 — Built-in decorators (@staticmethod, @property)

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def area(self):
        return 3.14 * self._radius ** 2

c = Circle(5)
print(c.area)   # accessed like an attribute, no parentheses
```

---

14. Regular Expressions

Definition: Regular expressions (regex) are patterns used to match, search, and manipulate text. Python's `re` module provides functions like `search()`, `match()`, `findall()`, and `sub()`.

Example 1 — Searching for a pattern

```
import re
text = "My email is test@example.com"
match = re.search(r"[\w.]+@[\w.]+", text)
print(match.group())   # test@example.com
```

Example 2 — Finding all matches

```
text = "Call 9876543210 or 9123456789"
numbers = re.findall(r"\d{10}", text)
print(numbers)   # ['9876543210', '9123456789']
```

Example 3 — Replacing text

```
text = "The rain in Spain"
new_text = re.sub(r"ain", "XYZ", text)
print(new_text)   # The rXYZ in SpXYZ
```

Example 4 — Validating a pattern

```
pattern = r"^[a-zA-Z0-9_]{3,16}$"   # username: 3-16 chars, letters/digits/underscore
username = "alex_99"
print(bool(re.match(pattern, username)))   # True
```

---

15. Dates & Time

Definition: The `datetime` module lets you work with dates, times, and time differences.

Example 1 — Current date and time

```
from datetime import datetime
now = datetime.now()
print(now)
print(now.strftime("%Y-%m-%d %H:%M:%S"))
```

Example 2 — Creating a specific date

```
from datetime import date
d = date(2026, 7, 3)
print(d)          # 2026-07-03
print(d.year, d.month, d.day)
```

Example 3 — Date arithmetic with timedelta

```
from datetime import datetime, timedelta
today = datetime.now()
next_week = today + timedelta(days=7)
print(next_week)
```

Example 4 — Parsing a date string

```
from datetime import datetime
date_str = "2026-07-03"
parsed = datetime.strptime(date_str, "%Y-%m-%d")
print(parsed)
```

---

16. Multithreading & Multiprocessing

Definition: Multithreading runs multiple threads concurrently within the same process — good for I/O-bound tasks (like network requests). **Multiprocessing** runs separate processes with their own memory — good for CPU-bound tasks, bypassing Python's Global Interpreter Lock (GIL).

Example 1 — Basic thread

```
import threading

def task():
    print("Task running")

t = threading.Thread(target=task)
t.start()
t.join()
```

Example 2 — Multiple threads

```
import threading

def print_numbers():
    for i in range(3):
        print(i)

threads = [threading.Thread(target=print_numbers) for _ in range(2)]
for t in threads:
    t.start()
for t in threads:
    t.join()
```

Example 3 — Basic multiprocessing

```
from multiprocessing import Process

def square(n):
    print(n * n)

if __name__ == "__main__":
    p = Process(target=square, args=(5,))
    p.start()
    p.join()
```

17. Databases (SQLite Basics)

Definition: Python's built-in `sqlite3` module lets you create, query, and manage a lightweight SQL database without installing external software.

Example 1 — Creating a table

```
import sqlite3
conn = sqlite3.connect("mydb.db")
cur = conn.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER, name TEXT)")
conn.commit()
```

Example 2 — Inserting data

```
cur.execute("INSERT INTO users VALUES (1, 'Alex')")
cur.execute("INSERT INTO users VALUES (2, 'Bob')")
conn.commit()
```

Example 3 — Querying data

```
cur.execute("SELECT * FROM users")
rows = cur.fetchall()
for row in rows:
    print(row)
```

Example 4 — Using parameters (avoids SQL injection)

```
name = "Charlie"
cur.execute("INSERT INTO users (id, name) VALUES (?, ?)", (3, name))
conn.commit()
conn.close()
```

18. Virtual Environments & pip

Definition: A **virtual environment** is an isolated Python setup that keeps a project's dependencies separate from your system-wide Python installation, preventing version conflicts. `pip` is Python's package manager used to install third-party libraries.

Example 1 — Creating and activating an environment

```
python -m venv myenv
myenv\Scripts\activate      # Windows
source myenv/bin/activate   # Mac/Linux
```

Example 2 — Installing packages

```
pip install requests
pip install pandas numpy matplotlib
```

Example 3 — Saving and reusing dependencies

```
pip freeze > requirements.txt
pip install -r requirements.txt
```

Example 4 — Deactivating the environment

```
deactivate
```

19. Popular Libraries

Definition: Beyond the standard library, Python has a rich ecosystem of third-party packages for data science, web development, automation, and more.

Library	Purpose
NumPy	Numerical computing, arrays
Pandas	Data analysis, dataframes
Matplotlib / Seaborn	Data visualization
Requests	Making HTTP requests / API calls
Flask / Django	Web development
BeautifulSoup / Scrapy	Web scraping
Scikit-learn / TensorFlow	Machine Learning

Example 1 — NumPy

```
import numpy as np
arr = np.array([1, 2, 3])
print(arr * 2)          # [2 4 6]
print(arr.mean())      # 2.0
```

Example 2 — Pandas

```
import pandas as pd
data = {"Name": ["Alex", "Bob"], "Age": [25, 30]}
df = pd.DataFrame(data)
print(df)
print(df["Age"].mean())
```

Example 3 — Requests (calling an API)

```
import requests
response = requests.get("https://api.github.com")
print(response.status_code)   # 200
print(response.json())
```

Example 4 — Flask (minimal web app)

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello, Flask!"

if __name__ == "__main__":
    app.run(debug=True)
```

20. Projects & Practice

Definition: Applying everything you've learned by building real, small projects is the fastest way to cement your Python skills.

Suggested build order: 1. Calculator (CLI) 2. To-Do list app 3. Number guessing game 4. Contact book (dictionaries + file storage) 5. Password generator 6. Weather app (using an API with `requests`) 7. Web scraper (BeautifulSoup) 8. Student management system (OOP + SQLite) 9. Simple Flask web app 10. Data analysis mini-project with Pandas + Matplotlib

Example — Mini number guessing game

```
import random

secret = random.randint(1, 10)
guess = None

while guess != secret:
    guess = int(input("Guess a number (1-10): "))
    if guess < secret:
        print("Too low!")
    elif guess > secret:
        print("Too high!")
    else:
        print("Correct! 🎉")
```

Suggested Study Plan

Week	Focus
1	Basics, variables, operators, control flow
2	Strings, lists, tuples, sets, dictionaries
3	Functions, OOP basics
4	Exception handling, file handling, modules
5	Comprehensions, generators, decorators
6	Regex, datetime, virtual environments, mini projects
7-8	Pick a track: Data Science (NumPy/Pandas) OR Web Dev (Flask/Django)
Ongoing	Solve 2-3 problems daily on HackerRank/LeetCode

Tip: Watch a topic on YouTube → immediately write the code yourself → solve 2-3 practice problems on that topic before moving to the next.